Python for AI & Machine Learning

Class 20 – Introduction to NumPy for AI & Machine Learning

Course: Python for AI & Machine Learning
Module: Data Science & Numerical Computing
Duration: 2 Hours (Theory + Practical)

Learning Objectives

By the end of this session, students will be able to:

- Understand what NumPy is and why it is used.
- Install and import the NumPy library.
- Create one-dimensional and multi-dimensional arrays.
- Perform mathematical operations on NumPy arrays.
- Understand the advantages of NumPy over Python Lists.
- Learn basic array indexing and slicing.
- Apply NumPy concepts in AI & Machine Learning projects.

Revision

In the previous class, we learned:

- Modules and Packages
- Built-in Modules
- User-defined Modules
- Third-party Modules
- The `math`, `random`, `datetime`, `os`, and `sys` modules
- Installing libraries using `pip`

Today, we begin one of the most important libraries used in Artificial Intelligence and Machine Learning—**NumPy**.

Introduction to NumPy

NumPy stands for **Numerical Python**.

It is an open-source Python library used for:

- Numerical Computing
- Scientific Computing
- Mathematical Operations
- Matrix Operations
- Array Processing
- Data Analysis
- Machine Learning

NumPy is the foundation of many AI and Data Science libraries, including:

- Pandas
- Scikit-learn
- TensorFlow
- Keras
- OpenCV
- SciPy

Why Do We Need NumPy?

Python Lists are useful for storing data, but they have limitations:

- Slower execution
- Higher memory usage
- Limited mathematical operations
- Not optimized for numerical computing

NumPy solves these problems by providing:

- Faster execution
- Less memory usage
- Powerful mathematical functions
- Efficient handling of large datasets
- Multi-dimensional arrays

Features of NumPy

- High Performance
- Multi-dimensional Arrays
- Fast Mathematical Computations
- Broadcasting Support
- Built-in Statistical Functions
- Matrix Operations
- Integration with AI & ML Libraries

Installing NumPy

Use the following command in the terminal or command prompt:

```bash id="9n8lxe" pip install numpy

```
To verify the installation:

```bash id="3twjlwm"
pip show numpy
```

Importing NumPy

```python id="qjdyb6" import numpy as np

```
`np` is an alias that makes NumPy functions easier to access.

Creating a NumPy Array

Example 1 – One-Dimensional Array

```python id="mjlwm8"
import numpy as np

numbers = np.array([10, 20, 30, 40, 50])

print(numbers)
```

Output

```text id="74h8h7" [10 20 30 40 50]

```

Creating a Two-Dimensional Array

```python id="fg3cju"
import numpy as np

matrix = np.array([
    [10, 20, 30],
    [40, 50, 60]
```

```
])

print(matrix)
```

Output

```text id="fcjlwm" [[10 20 30] [40 50 60]]

```

Creating a Three-Dimensional Array

```python id="rh28k9"
import numpy as np

array3d = np.array([
    [[1, 2], [3, 4]],
    [[5, 6], [7, 8]]
])

print(array3d)
```

Difference Between Python List and NumPy Array

Python List	NumPy Array
Slower	Faster
Uses more memory	Uses less memory
Limited mathematical operations	Powerful mathematical operations
Suitable for general-purpose programming	Optimized for scientific and numerical computing
Supports mixed data types	Best performance with homogeneous data types

---

Checking Array Properties

Shape

Returns the dimensions of an array.

```python id="wrblqe" import numpy as np
```

arr = np.array([[1, 2], [3, 4]])

print(arr.shape)

```
### Output

```text id="rmtwzb"
(2, 2)
```

Number of Dimensions

```python id="ccjlwm" print(arr.ndim)

```
Output

```text id="q8eqqg"
2
```

Data Type

```python id="5jlwm4" print(arr.dtype)

```

Size

Returns the total number of elements.

```python id="7jlwm9"
print(arr.size)
```

Output

```text id="6jlwm2" 4

```

Creating Special Arrays

Array of Zeros

```python id="1jlwm8"

```
import numpy as np

zeros = np.zeros((3, 3))

print(zeros)
```

Array of Ones

```python id="9jlwm0" ones = np.ones((2, 4))

print(ones)

```

Identity Matrix

```python id="jlwm45"
identity = np.eye(4)

print(identity)
```

Range of Numbers

```python id="jlwm21" numbers = np.arange(1, 11)

print(numbers)

```
Output

```text id="jlwm22"
[1 2 3 4 5 6 7 8 9 10]
```

Evenly Spaced Numbers

```python id="jlwm23" numbers = np.linspace(0, 100, 11)

print(numbers)

```
Output

```text id="jlwm24"
[  0.  10.  20.  30.  40.  50.  60.  70.  80.  90. 100.]
```

Basic Mathematical Operations

```python id="jlwm25" import numpy as np

a = np.array([10, 20, 30])

b = np.array([1, 2, 3])

print(a + b) print(a - b) print(a * b) print(a / b)
```

Output

```text id="jlwm26"
[11 22 33]
[ 9 18 27]
[10 40 90]
[10. 10. 10.]
```

Aggregate Functions

```python id="jlwm27" numbers = np.array([10, 20, 30, 40, 50])

print(np.sum(numbers)) print(np.mean(numbers)) print(np.max(numbers)) print(np.min(numbers))
```

Output

```text id="jlwm28"
150
30.0
50
10
```

Array Indexing

```python id="jlwm29" numbers = np.array([100, 200, 300, 400])

print(numbers[0]) print(numbers[2])
```

```
### Output

```text id="jlwm30"
100
300
```

Array Slicing

```python id="jlwm31" numbers = np.array([10, 20, 30, 40, 50])

print(numbers[1:4])
```

```
### Output

```text id="jlwm32"
[20 30 40]
```

AI Example – Student Marks Analysis

```python id="jlwm33" import numpy as np

marks = np.array([85, 92, 78, 95, 88])

print("Average:", np.mean(marks)) print("Highest:", np.max(marks)) print("Lowest:", np.min(marks))
```

```
---

# AI Example – Image Pixel Matrix

```python id="jlwm34"
import numpy as np

image = np.array([
    [255, 120, 90],
    [100, 80, 60],
    [255, 255, 255]
])

print(image)
```

Images are represented as NumPy arrays in computer vision.

```

AI Example – Sensor Data

```python id="jlwm35" import numpy as np

temperature = np.array([30.5, 31.2, 29.8, 30.9])

print("Average Temperature:", np.mean(temperature)) ```

---

# Advantages of NumPy

- Faster than Python Lists.
- Efficient memory management.
- Easy mathematical operations.
- Excellent support for matrices.
- Highly optimized for numerical computing.
- Used by almost every AI and Data Science library.

---

# Practical Exercise 1

Create the following arrays:

- One-dimensional array
- Two-dimensional array
- Three-dimensional array

Display:

- Shape
- Dimensions
- Size
- Data Type

---

# Practical Exercise 2

Create:

- Zero Matrix
- One Matrix
- Identity Matrix

Display all matrices.

---
```

Practical Exercise 3

Perform:

- Addition
- Subtraction
- Multiplication
- Division

Using two NumPy arrays.

Practical Exercise 4

Create an array of student marks.

Calculate:

- Total
- Average
- Highest Marks
- Lowest Marks

Practical Exercise 5

Create an array using:

- `arange()`
- `linspace()`

Compare the outputs.

Assignment

Student Performance Analysis using NumPy

Develop a program that:

- Stores marks of 10 students in a NumPy array.
- Calculates:
- Total Marks
- Average Marks
- Highest Marks
- Lowest Marks
- Number of students scoring above 75.

- Display all results clearly.

Bonus Challenge:

Create a 5 × 5 matrix representing classroom seating and display its shape, size, and dimensions.

Interview Questions

1. What is NumPy?
2. Why is NumPy faster than Python Lists?
3. What are the advantages of NumPy?
4. What is the difference between a Python List and a NumPy Array?
5. What is an ndarray?
6. What is the purpose of `np.array()`?
7. Explain the use of `zeros()`, `ones()`, `eye()`, `arange()`, and `linspace()`.
8. What is the difference between `shape`, `size`, and `ndim`?
9. How is NumPy used in AI and Machine Learning?
10. Name five libraries that depend on NumPy.

Summary

In today's session, we learned:

- Introduction to NumPy
- Installing and Importing NumPy
- Creating Arrays
- One-Dimensional, Two-Dimensional, and Three-Dimensional Arrays
- Array Properties (`shape`, `size`, `ndim`, `dtype`)
- Creating Special Arrays
- Mathematical Operations
- Aggregate Functions
- Array Indexing and Slicing
- AI & Machine Learning Applications of NumPy
- Practical Exercises
- Assignment

NumPy is the backbone of scientific computing in Python. Mastering NumPy is essential before learning **Pandas**, **Matplotlib**, **Scikit-learn**, and **Deep Learning frameworks**.

Next Class Preview

Class 21 – Advanced NumPy Operations

Topics include:

- Array Reshaping
- Flattening Arrays
- Resizing Arrays
- Broadcasting
- Boolean Indexing
- Fancy Indexing
- Sorting Arrays
- Searching Arrays
- Stacking and Splitting Arrays
- Vectorization
- Real-world AI & Machine Learning examples